LOCALCONNECT

PHASE 4.6 — PROGRAM DESIGN

Program Pseudocode

System Design Phase | IT Group Project

Purpose: This section describes the planned logic of the LocalConnect service marketplace. It presents design-level pseudocode for the principal workflows and is intended to guide implementation and testing. It is not production source code.

Technology alignment: Flutter client • Node.js/Express API • Supabase Auth and PostgreSQL • Google Maps integration • PayFast payment integration • Push notifications

4.6 Program Design (Program Pseudocode)

The program design translates the proposed LocalConnect requirements into structured algorithms. The client application provides the user interface, while the Express API enforces validation, permissions and business rules. Supabase services provide authentication and database access. External integrations support location-based discovery, payments and notifications.

The pseudocode below describes the principal processes for customers, service providers and administrators. It uses a modular-monolith backend structure: routes receive requests, controllers coordinate operations, service modules apply business rules, repositories access data, and integration modules communicate with external services.

4.6.1 System Startup and Request Handling

This process starts the backend, registers the API routes and applies common checks to incoming requests before they reach the relevant business logic.

```
PROGRAM StartLocalConnectServer
    LOAD environment configuration
    VERIFY required configuration values are present
    INITIALISE connection to Supabase and PostgreSQL
    REGISTER security headers, request logging and body parsing
    REGISTER authentication and input-validation middleware
    REGISTER routes for authentication, profiles, provider search,
        services, bookings, payments, messages, reviews,
        notifications and administration
    REGISTER central error handler
    START HTTP server on configured port
END PROGRAM

HANDLER HandleAPIRequest(request)
    route <- FIND route matching request method and path

    IF route does not exist THEN
        RETURN 404 Not Found
    END IF

    IF route requires authentication THEN
        token <- EXTRACT bearer token from request
        identity <- VERIFY token using Supabase Auth
        IF identity is invalid THEN
            RETURN 401 Unauthorised
        END IF
        user <- LOAD application profile for identity
    END IF

    IF route has role restrictions AND user role is not permitted THEN
        RETURN 403 Forbidden
    END IF

    IF request parameters or body fail validation THEN
        RETURN 400 Bad Request with safe validation details
    END IF

    result <- CALL route controller with validated request
    RETURN appropriate success response containing result
END HANDLER
```

4.6.2 User Registration, Login and Access Control

Registration creates an authenticated account and a corresponding application profile. Login verifies credentials and returns a valid session only when the account meets the configured verification requirements. Role and ownership checks protect restricted operations.

```
PROCEDURE RegisterUser(email, password, profileData, requestedRole)
    VALIDATE email format, password policy and required profile fields
    IF requestedRole is not an allowed public role THEN
        RETURN 400 Invalid Role
    END IF

    authUser <- CREATE account using Supabase Auth
```

```
    IF account creation fails THEN
        RETURN safe registration error
    END IF

    CREATE application profile linked to authUser
    SET profile role to requestedRole
    SET account status to active or pending verification as required
    SEND verification email when enabled
    RETURN account-created response and verification instructions
END PROCEDURE

PROCEDURE LoginUser(email, password)
    session <- AUTHENTICATE credentials using Supabase Auth
    IF authentication fails THEN
        RETURN 401 Invalid Credentials
    END IF

    profile <- LOAD application profile for session user
    IF profile does not exist OR account is suspended THEN
        INVALIDATE session when supported
        RETURN access denied
    END IF

    IF email verification is required AND not complete THEN
        RETURN verification-required response
    END IF

    RETURN session tokens and permitted profile information
END PROCEDURE

PROCEDURE AuthoriseAction(request, permittedRoles, resourceOwnerId)
    identity <- VERIFY access token
    IF identity is invalid or expired THEN
        RETURN 401 Unauthorised
    END IF

    profile <- LOAD profile for identity
    IF profile role is not in permittedRoles THEN
        RETURN 403 Forbidden
    END IF

    IF resourceOwnerId is supplied AND
       identity user ID is not resourceOwnerId AND profile role is not admin THEN
        RETURN 403 Forbidden
    END IF

    RETURN authorised identity and profile
END PROCEDURE
```

4.6.3 Service Provider Discovery

Customers can search for available service providers by location and apply filters such as service category, price and rating. The server validates search criteria and returns only providers eligible for public listing.

```
PROCEDURE SearchServiceProviders(location, radius, filters, page)
    VALIDATE location, radius, filters and page values
    IF any required search value is invalid THEN
        RETURN 400 Invalid Search Criteria
    END IF

    IF location is an address THEN
        coordinates <- GEOCODE address using Google Maps
        IF geocoding fails THEN
            RETURN location-not-found response
        END IF
    ELSE
        coordinates <- VALIDATE supplied coordinates
    END IF

    providers <- QUERY active, publicly listed and verified providers
                 whose active services match the search area
    providers <- APPLY category, price and rating filters
    providers <- SORT by selected option, such as distance or rating
    results <- PAGINATE providers using validated page settings
```

```
    FOR EACH provider IN results
        CALCULATE or retrieve distance from coordinates
        ADD travel estimate when the service is available
    END FOR

    RETURN results and pagination metadata
END PROCEDURE
```

4.6.4 Booking Management

Booking algorithms validate service availability and prevent conflicting reservations. The database must enforce suitable transaction or locking rules so two customers cannot successfully reserve the same provider time slot.

```
PROCEDURE RequestBooking(customer, providerId, serviceId,
                         scheduledStart, notes)
    REQUIRE authenticated customer role
    VALIDATE service exists, is active and belongs to providerId
    VALIDATE scheduledStart is in the future
    VALIDATE requested time is within provider availability
    price <- READ current price for the selected service

    BEGIN DATABASE TRANSACTION
        LOCK or atomically reserve provider's requested time interval
        IF an active booking conflicts with the interval THEN
            ROLLBACK TRANSACTION
            RETURN 409 Time Slot Unavailable
        END IF

        booking <- INSERT booking with customer, provider, service,
                   scheduledStart, price, notes,
                   booking status = pending,
                   payment status = pending
    COMMIT TRANSACTION

    SEND new-booking notification to provider
    RETURN booking details
END PROCEDURE

PROCEDURE RespondToBooking(provider, bookingId, action, reason)
    booking <- LOAD booking
    REQUIRE booking belongs to provider
    REQUIRE booking status is pending

    IF action = accept THEN
        CONFIRM availability using an atomic database operation
        IF time slot is no longer available THEN
            RETURN 409 Time Slot Unavailable
        END IF
        SET booking status to confirmed
    ELSE IF action = decline THEN
        SET booking status to declined
        SAVE decline reason when provided
    ELSE
        RETURN 400 Invalid Action
    END IF

    SAVE booking status change
    NOTIFY customer of the decision
    RETURN updated booking
END PROCEDURE

PROCEDURE RescheduleBooking(customer, bookingId, newScheduledStart)
    booking <- LOAD booking
    REQUIRE customer is authorised to modify this booking
    REQUIRE booking is eligible for rescheduling
    VALIDATE new time is in the future and within provider availability

    BEGIN DATABASE TRANSACTION
        LOCK or atomically reserve the new time interval
        IF new interval conflicts with another active booking THEN
            ROLLBACK TRANSACTION
            RETURN 409 Time Slot Unavailable
        END IF
        UPDATE booking scheduled time
        SET booking status to pending for provider reconfirmation
```

```
    COMMIT TRANSACTION

    NOTIFY provider that reconfirmation is required
    RETURN updated booking
END PROCEDURE

PROCEDURE CancelBooking(actor, bookingId, reason)
    booking <- LOAD booking
    REQUIRE actor is customer or provider associated with booking
    REQUIRE booking is not completed, cancelled or declined
    CHECK cancellation policy and any applicable restrictions
    SET booking status to cancelled
    SAVE cancellation reason and actor
    RELEASE reserved time slot
    NOTIFY the other booking participant
    RETURN updated booking
END PROCEDURE

PROCEDURE CompleteBooking(provider, bookingId)
    booking <- LOAD booking
    REQUIRE provider owns the booking
    REQUIRE booking status is confirmed or in progress
    SET booking status to completed
    SAVE status change
    NOTIFY customer that a review may be submitted
    RETURN updated booking
END PROCEDURE
```

4.6.5 Payment Processing and PayFast Confirmation

The application initiates payments through a hosted PayFast checkout flow. A payment is only marked as successful after the backend validates the server-to-server confirmation. Callback handling must be idempotent so repeated notifications do not duplicate payment effects.

```
PROCEDURE InitiatePayment(customer, bookingId)
    booking <- LOAD booking
    REQUIRE customer owns the booking
    REQUIRE booking status is confirmed
    REQUIRE no successful payment already exists for booking

    payment <- CREATE payment record with pending status,
               expected amount and currency
    redirectUrl <- CREATE PayFast payment request using server-side
                   configuration and the payment reference
    RETURN payment reference, amount and redirectUrl
END PROCEDURE

HANDLER ProcessPayFastNotification(notification)
    VALIDATE required callback fields
    IF fields are missing or malformed THEN
        RECORD safe callback error
        RETURN callback response required by PayFast
    END IF

    VERIFY callback signature and required source checks
    IF verification fails THEN
        RECORD rejected callback without exposing secrets
        RETURN callback response required by PayFast
    END IF

    BEGIN DATABASE TRANSACTION
        payment <- LOAD referenced payment with a row lock
        IF payment does not exist THEN
            RECORD callback for investigation
            COMMIT TRANSACTION
            RETURN callback acknowledgement
        END IF

        IF payment status is already final THEN
            COMMIT TRANSACTION
            RETURN callback acknowledgement
        END IF

        IF callback amount or currency differs from expected values THEN
            RECORD payment discrepancy
```

```
            ROLLBACK TRANSACTION
            RETURN callback acknowledgement
        END IF

        IF verified callback status indicates completion THEN
            SET payment status to paid
            SET linked booking payment status to paid
        ELSE IF verified callback status indicates failure THEN
            SET payment status to failed
        ELSE
            RECORD callback as pending or unrecognised for review
        END IF
    COMMIT TRANSACTION

    NOTIFY relevant users of the confirmed payment outcome
    RETURN callback acknowledgement
END HANDLER
```

4.6.6 Messaging and Reviews

Messaging is limited to authorised conversation participants. Reviews are linked to completed bookings so that users cannot submit multiple reviews for the same booking.

```
PROCEDURE SendMessage(sender, conversationId, messageText)
    REQUIRE sender is authenticated
    REQUIRE sender is a participant in conversationId
    VALIDATE messageText is not empty and is within size limits
    IF validation fails THEN
        RETURN 400 Invalid Message
    END IF

    message <- SAVE message to database
    PUBLISH message to authorised real-time channel when available
    CREATE recipient notification according to preferences
    RETURN saved message
END PROCEDURE

PROCEDURE SubmitReview(customer, bookingId, rating, reviewText)
    booking <- LOAD booking
    REQUIRE customer owns the booking
    REQUIRE booking status is completed
    REQUIRE no review already exists for this booking
    VALIDATE rating is an integer from 1 through 5
    VALIDATE reviewText against configured length and content rules

    review <- SAVE review with applicable visibility or moderation status
    RECALCULATE provider's displayed average rating
    RETURN saved review
END PROCEDURE

PROCEDURE ModerateReview(admin, reviewId, decision, notes)
    REQUIRE admin role
    REQUIRE decision is hide or restore
    review <- LOAD review
    UPDATE review visibility
    RECORD administrator, decision, timestamp and notes in audit log
    RETURN updated review
END PROCEDURE
```

4.6.7 Provider Verification and Administration

Provider verification helps ensure that only providers who meet the platform's configured requirements are publicly listed. Administrative operations must be restricted to authorised administrators and recorded for accountability.

```
PROCEDURE SubmitProviderVerification(provider, documents)
    REQUIRE provider role
    VALIDATE required document types and file constraints
    STORE documents in protected storage
    CREATE verification submission with status pending
    NOTIFY administrators of the submission
    RETURN submission reference and status
END PROCEDURE
```

```
PROCEDURE ReviewProviderVerification(admin, submissionId, decision, notes)
    REQUIRE admin role
    submission <- LOAD verification submission
    REQUIRE submission status is pending
    REQUIRE decision is approve or reject

    IF decision = approve THEN
        SET submission status to approved
        MARK provider verified and eligible for public listing
    ELSE
        SET submission status to rejected
        KEEP provider hidden from public search
        SAVE rejection outcome or notes
    END IF

    RECORD reviewer and decision in audit log
    NOTIFY provider of the outcome
    RETURN updated submission
END PROCEDURE

PROCEDURE SuspendAccount(admin, userId, reason)
    REQUIRE admin role
    REQUIRE reason is provided
    user <- LOAD target account
    IF user does not exist THEN
        RETURN 404 Account Not Found
    END IF

    SET account status to suspended
    REVOKE or invalidate active sessions where supported
    RECORD administrative action in audit log
    NOTIFY affected user when appropriate
    RETURN updated account status
END PROCEDURE
```

4.6.8 Notifications and Error Handling

Notifications communicate important booking, payment, message and account events. Errors are logged for diagnosis, while responses to users must avoid exposing credentials, tokens, personal data or internal implementation details.

```
PROCEDURE NotifyUser(userId, eventType, details)
    user <- LOAD recipient account
    preferences <- LOAD notification preferences for eventType

    IF eventType is mandatory under system policy OR
       preferences allow this notification THEN
        SAVE in-app notification
    END IF

    IF push notifications are enabled and permitted THEN
        SEND push notification through configured provider
    END IF

    RECORD delivery outcome without logging secrets
    RETURN notification processing result
END PROCEDURE

HANDLER HandleApplicationError(error, request)
    correlationId <- GENERATE request correlation identifier
    WRITE structured diagnostic details to restricted server log

    IF error is validation error THEN
        RETURN 400 with safe validation message and correlationId
    ELSE IF error is unauthorised THEN
        RETURN 401 with safe message and correlationId
    ELSE IF error is forbidden THEN
        RETURN 403 with safe message and correlationId
    ELSE IF error is not found THEN
        RETURN 404 with safe message and correlationId
    ELSE IF error is a booking or state conflict THEN
        RETURN 409 with safe message and correlationId
    ELSE
        RETURN 500 with generic error message and correlationId
    END IF
```

```
    NEVER return secrets, access tokens or stack traces to the client
END HANDLER
```

4.6.9 Design Considerations

• The backend is the authoritative layer for permissions, validation, prices, booking states and payment states; client-side checks support usability but do not replace server-side validation.

• Database transactions and atomic operations should protect booking availability and payment updates from concurrent requests.

• Every state-changing operation must verify the caller's role and their relationship to the affected resource.

• Payment notifications must be verified and processed idempotently; a browser redirect alone must not be treated as proof of payment.

• Secrets and integration credentials must be stored in server-side environment configuration or an appropriate secret-management service, not embedded in the Flutter application.

• External-service failures must not cause the application to falsely confirm a booking or payment. The system should record the issue and return a controlled response.

• The pseudocode describes intended design behaviour. Specific details such as cancellation rules, provider document requirements, payment currencies and notification preferences must match the final approved requirements and implementation.

End of Section 4.6

This program design provides the algorithmic basis for implementing and testing LocalConnect's main workflows. The final implementation should be checked against the approved requirements, database schema, interface designs and security design in the other subsections of Phase 4.